

SOFTWARE IMPLEMENTATION PROCEDURE

RENTAMOVIL

**Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana Ramírez
Vargas**

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

1. DOCUMENT OBJECTIVE

Clearly and verifiably establish the technical and operational steps followed to deploy, configure, and launch the RentaMóvil application (Web and Mobile) so that it is publicly accessible, documenting exactly what was executed and verified as working—not an aspirational plan.

2. SCOPE

This procedure covers everything from infrastructure preparation (Docker, shared network, reverse proxy) to the public release of the web frontend and API, and distribution of an Android installable package for the mobile app.

2.1 Scope decision: own server + tunnel, not cloud or app stores

By team decision, the deployment uses an owned Linux machine as the server (not a cloud provider), with Docker + nginx as the reverse proxy + ngrok for the public URL. This avoids depending on a cloud host with SQL Server support (the project's actual database engine, which is not included free of charge in most PaaS offerings) and is consistent with the project's academic scope. The mobile app is distributed as a directly installable APK (EAS Build, internal profile), without going through Google Play / App Store—avoiding developer account costs and external review times (risk R-004 in rtm-docs/15-project-control/risks.md).

2.2 Deployed system architecture

- **Web Frontend:** React + Vite, served in production ("vite preview") behind nginx.
- **Mobile Frontend:** Expo + React Native, compiled into an Android .apk with EAS Build ("preview" profile, internal distribution).
- **Backend:** 4 Spring Boot microservices (rtm-iam, rtm-fleet-maintenance, rtm-booking-reservation, rtm-payment-billing) behind a single API Gateway (rtm-api-gateway).
- **Database:** a single shared SQL Server instance, with one schema per service (ADR-009).
- **Release:** nginx as the single reverse proxy + ngrok for the public URL with a reserved fixed domain.

3. IMPLEMENTATION PHASES (EXECUTED AND VERIFIED)

Phase 1 — Environment and infrastructure preparation

Step 1.1: Shared Docker network

A single external Docker network was created and is used by all 11 repositories to discover one another by service name:

```
docker network create rentamovil
```

Step 1.2: ngrok account and domain

A free ngrok account was created and a fixed domain was reserved (Dashboard → Domains → "+ New Domain"), so the public URL does not change when the tunnel restarts.

Phase 2 — Deployment of databases and backend services

Each repository was started in the actual dependency order (foreign keys and shared schemas), with verification through the gateway's /health endpoint after each step:

#	Repository	Action
1	rtm-iam-db	docker compose up -d – starts the shared SQL Server, creates the rentamovil database, and migrates the iam schema
2	rtm-fleet-maintenance-db	docker compose up -d – migrates the fleet schema (depends on shared.audit, created by iam-db)
3	rtm-booking-db	docker compose up -d – migrates the booking schema (references fleet.vehicle / fleet.branch)
4	rtm-payment-billing-db	docker compose up -d – migrates the payment schema (references booking.reservation)
5	rtm-iam	docker compose up -d – identity and access service
6	rtm-fleet-maintenance	docker compose up -d – fleet and maintenance service
7	rtm-booking-reservation	docker compose up -d – reservation service
8	rtm-payment-billing	docker compose up -d – payment service
9	rtm-api-gateway	docker compose up -d – routes requests to the 4 services above, public port 8080
10	RentaMovil/web/Front-end	docker compose up -d --build – production frontend build, port 4173
11	rtm-deploy	docker compose up -d – nginx (reverse proxy, port 80) + ngrok (public tunnel with fixed domain)

Verification performed after step 9:

```
curl http://localhost:8080/health
→ {"status":"degraded","upstreams":{"iam":"ok","fleet-maintenance":"ok",
  "booking-reservation":"ok","payment-billing":"ok","telemetry-gps":"down"}}
```

"degraded" is the correct and expected status: telemetry-gps is a service that rtm-docs documents as a future goal and that does not yet exist as a repository (see 01-MAPA-GLOBAL-DEL-PROYECTO.md in the study notes); the 4 actual services in the current scope return "ok".

Phase 3 — Publishing the web frontend and API behind a single domain

Step 3.1: nginx routing

The web frontend already sends its API calls to `/api` on its own origin by default (the same pattern as Vite's development proxy), so nginx simply reproduces that routing scheme without changing the frontend or API code:

Public path	Destination (container)	Purpose
<code>/</code>	web (:4173)	Web frontend — production build served with "vite preview"
<code>/api/*</code>	api-gateway (:8080), without the <code>/api</code> prefix	Everything that already has a real backend: auth, users, vehicles, branches, reservations, payments, bank accounts
<code>/mock/*</code>	mock-api (:3100), without the <code>/mock</code> prefix	Features that do not yet have a real service (see <code>REAL_BACKEND_PREFIXES</code> in <code>httpClient.js</code>)

Step 3.2: Connectivity verification (performed)

```
curl https://commerce-playmate-carving.ngrok-free.dev/  
→ HTTP 200 (actual frontend index.html)  
  
curl https://commerce-playmate-carving.ngrok-free.dev/api/health  
→ identical to the direct curl request to :8080/health
```

Actual incident during this phase (resolved)

Vite "allowedHosts" block:

When the public URL was opened for the first time, "vite preview" returned 403 "Blocked request... not allowed" because, by default, it only accepts requests with a localhost Host. This was fixed by adding `allowedHosts: ['.ngrok-free.dev', '.ngrok.app', '.ngrok.io']` to the preview section of `vite.config.js` and rebuilding the web container.

Al abrir la URL pública por primera vez, "vite preview" devolvía 403 "Blocked request... not allowed" porque por defecto solo acepta peticiones con Host de localhost. Se corrigió agregando `allowedHosts: ['.ngrok-free.dev', '.ngrok.app', '.ngrok.io']` a la sección preview de `vite.config.js`, y reconstruyendo el contenedor web.

Phase 4 — Building and distributing the mobile app (APK)

Step 4.1: EAS project configuration

```
npx eas-cli login  
npx eas-cli build:configure # creates eas.json and links the project to the Expo account
```

The "preview" profile in `eas.json` was adjusted to generate a directly installable `.apk` (Android compiles to `.aab` by default, which is only suitable for uploading to the Play Store):

```
"preview": { "distribution": "internal", "android": { "buildType": "apk" } }
```

Step 4.2: Environment variable with the public URL

The API URL is fixed inside the compiled binary—it was configured before building:

```
EXPO_PUBLIC_API_URL=https://commerce-playmate-carving.ngrok-free.dev/api
```

Step 4.3: Build

```
npx eas-cli build --platform android --profile preview
```

Actual incident during this phase (resolved)

Gradle failure due to a corrupted asset:

The first build failed during the "Run gradlew" phase with "AAPT: error: file failed to compile" for assets/images/login.png. The cause: the file was actually a WebP image with a .png extension (the actual format did not match the extension), which the Android resource compiler (aapt2) rejects. It was fixed by converting the file to a real PNG (same visual content, correct format), then restarting the build, which completed successfully.

El primer build falló en la fase "Run gradlew" con "AAPT: error: file failed to compile" sobre assets/images/login.png. La causa: el archivo era en realidad una imagen WebP con extensión .png (formato real distinto al declarado por el nombre), y el compilador de recursos de Android (aapt2) lo rechaza. Se corrigió convirtiendo el archivo a un PNG real (mismo contenido visual, formato correcto) y se relanzó el build, que completó exitosamente.

Step 4.4: Installation and verification

The .apk was downloaded from the link provided by EAS Build upon completion, installed on an Android device (with "unknown sources" enabled), and the application's real workflow was verified against the public backend—it worked correctly.

Evidence:

APK installed and application verified as working against the public backend (commerce-playmate-carving.ngrok-free.dev)—confirmed on October 9, 2026.

APK instalado y aplicación verificada funcionando contra el backend público (commerce-playmate-carving.ngrok-free.dev) — confirmado el 9 de octubre de 2026.

Phase 5 — Production smoke tests (completed)

- Gateway health check through nginx: 200, 4 actual services "ok".
- Web frontend loaded through the public URL: 200, actual content served.
- Full workflow in the installed mobile app (login, navigation, use of the real API) against the public backend: verified as working by the team.

4. RISK MITIGATION AND CONTINGENCY PLAN

The risks of this deployment are formally recorded in rtm-docs/15-project-control/risks.md. The most relevant to this implementation are:

- **R-003 — SQL Server is a single instance (no high availability): if it goes down, all 4 services stop responding. Mitigation applied: documented recovery procedure (docker compose**

down + up, not restart, in rtm-iam-db)—already executed once during this same implementation.

- **Non-fixed public URL (ngrok, free plan)**—mitigated by reserving a fixed domain in the ngrok account and explicitly passing it to the container (`--url=...` in `rtm-deploy/docker-compose.yml`), so the URL does not change when the tunnel restarts.
- **R-004 — Rejection by mobile app stores: avoided by design**—the app is not published in stores within this scope; it is distributed as an internally signed APK through EAS Build.
- **APK with a fixed backend URL**—if the public domain changes, the `.apk` must be rebuilt and redistributed. This is documented as a known limitation, not a bug.